<!-- به نام خدا -->

مهندسی اینترنت، فصل ۱: پروتکل‌ها

زیربنای HTTP: اتصال و TLS

جلسهٔ ۴: چرا HTTP سه نسخه دارد، و TLS دقیقاً از چه چیزی محافظت می‌کند؟

parse · HSTS · DNS · **اتصال** · **TLS** · HTTP

محمد بصیرزاده

دانشگاه شهید چمران اهواز، دانشکدهٔ مهندسی، پاییز ۱۴۰۵

اگر پهنای باند دو برابر شود، صفحه چقدر زودتر باز می‌شود؟

اینترنت خانه‌تان از ۵ به ۱۰ مگابیت بر ثانیه رسیده است. با انگشت رأی دهید.

الف) دو برابر زودتر

ب) حدود ۵۰٪ زودتر

ج) حدود ۵٪ زودتر

د) بدون تغییر

اگر پهنای باند دو برابر شود، صفحه چقدر زودتر باز می‌شود؟

اینترنت خانه‌تان از ۵ به ۱۰ مگابیت بر ثانیه رسیده است. با انگشت رأی دهید.

الف) دو برابر زودتر

ب) حدود ۵۰٪ زودتر

ج) حدود ۵٪ زودتر

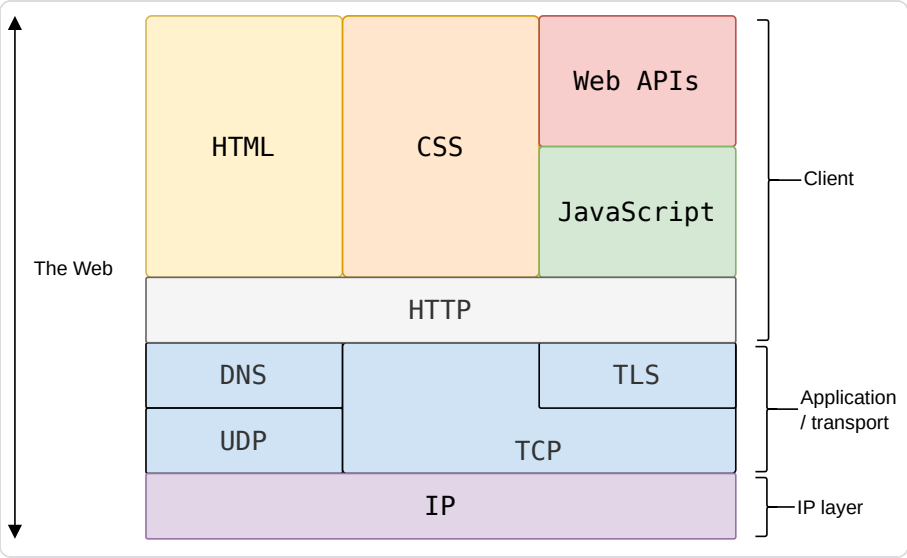
د) بدون تغییر

پاسخ: ج) در آزمایش Mike Belshe از Google، دو برابر شدن پهنای باند زمان بارگذاری را فقط حدود ۵٪ کم کرد، اما هر ۲۰ میلی‌ثانیه کاهش زمان رفت‌وبرگشت آن را ۷ تا ۱۵٪ کم کرد.

> *However, decreasing RTT, regardless of current bandwidth always helps make web browsing faster.*
>
> Mike Belshe, More Bandwidth Doesn't Matter (much), 2010

کندی صفحه بیشتر از رفت‌وبرگشت‌هاست تا از کمبود پهنای باند.

اتصال و TLS کجای پشتهٔ پروتکل‌ها قرار دارند؟



HTTP روی TLS، و TLS روی TCP

منبع: MDN, Mozilla Contributors · CC BY-SA 2.5

- TCP یا QUIC پیام HTTP را کامل و به ترتیب می‌رساند و TLS آن را رمز می‌کند
- پیش از اولین بایت HTTP دو handshake لازم است، یکی برای اتصال و یکی برای TLS، و هر کدام **دست‌کم یک RTT** طول می‌کشد

> *Round Trip Time (RTT) is the length of time it takes for a data packet to be sent to a destination plus the time it takes for an acknowledgment of that packet to be received back at the origin.*
>
> MDN, Glossary: RTT

در این جلسه هر نسخه را با یک عدد می‌سنجیم: تعداد RTT تا اولین بایت پاسخ.

صد میلی‌ثانیهٔ اول یک درخواست صرف چه می‌شود؟

سؤال از زمان یک درخواست HTTPS، چه سهمی صرف اتصال و TLS می‌شود و چه سهمی صرف خود HTTP؟

مشاهده فاصلهٔ ستون‌های پشت‌سرهم را برای یک سرور نزدیک و یک سرور دور مقایسه کنید؛ خط دوم همان درخواست است، روی همان اتصال.

تصمیم سرور را تا جای ممکن به کاربر نزدیک کنید و اتصال را دوباره به کار ببرید.

Linux / macOS | Windows

```
U=https://www.scu.ac.ir/
F='dns %{time_namelookup}  tcp %{time_connect}  '
F+='tls %{time_appconnect}  ttfb %{time_starttransfer}\n'
curl -s -o /dev/null -o /dev/null -w "$F" $U $U
```

هر عدد از لحظهٔ شروع همان درخواست شمرده می‌شود؛ ستون `ttfb` زمان رسیدن اولین بایت پاسخ است. درخواست دوم همان URL است، روی همان اتصال. سپس همین را برای `https://example.com/` اجرا کنید.

خروجی curl را چگونه بخوانیم؟

فاصلهٔ دو ستون	تقریباً برابر است با
از `dns` تا `tcp`	یک RTT برای TCP three-way handshake
از `tcp` تا `tls`	یک RTT در TLS 1.3، و دو RTT در TLS 1.2
از `tls` تا `ttfb`	یک RTT، به‌اضافهٔ زمان پردازش در سرور
در خط دوم، `tcp` و `tls` صفرند	اتصال دوباره به کار رفته و **فقط یک RTT** لازم است

- اگر شبکه از یک proxy عبور کند، ستون‌های tcp و tls زمان رسیدن به proxy را نشان می‌دهند، نه به سرور
- کد کامل و توضیح اجرا در `labs/04-connection-tls/timing` است

یک درخواست تازهٔ HTTPS چند RTT صبر می‌کند؟

پشتهٔ پروتکل	handshake اتصال	handshake TLS	Request / Response	جمع RTT
TCP + TLS 1.2	؟	؟	؟	؟
TCP + TLS 1.3	۱	۱	۱	۳
TCP + TLS 1.3 + 0-RTT	؟	؟	؟	؟
QUIC (HTTP/3)	؟		؟	؟
QUIC + 0-RTT	؟		؟	؟

تا رسیدن اولین بایت پاسخ، بدون DNS، و با فرض اینکه هر پیام در یک بسته جا می‌شود. بقیهٔ جدول را در طول جلسه کامل می‌کنیم.

سه RTT پیش از اولین بایت پاسخ؛ هدف این جلسه کم کردن همین عدد است.

بخش ۱ از ۵

اتصال در HTTP/1.x: یکی یا چندتا؟

تا اینجا: در دمو دیدیم که درخواست دوم روی همان اتصال قبلی، بدون handshake تازه، پاسخ گرفت.

سؤال این بخش

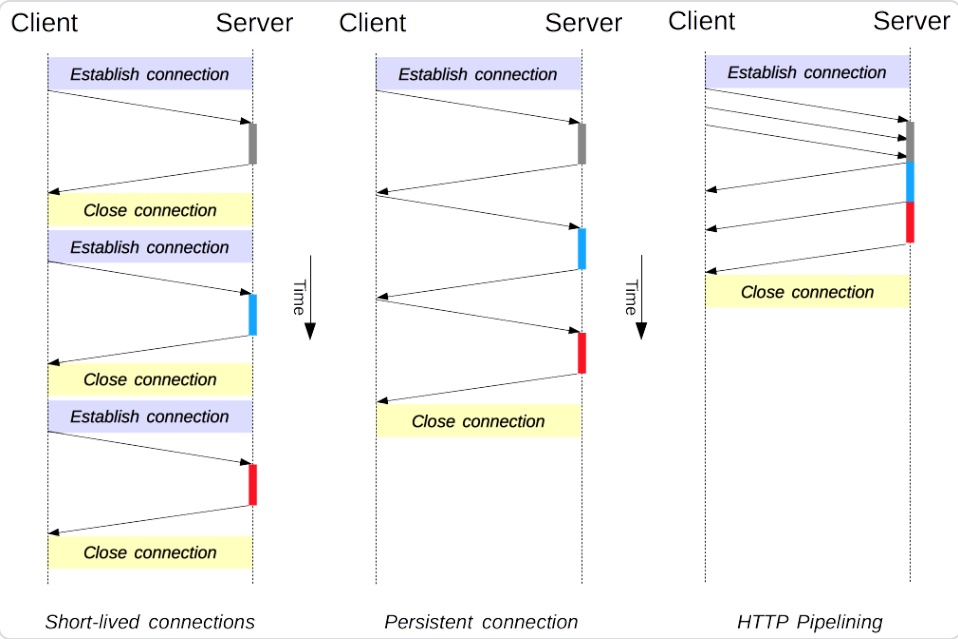
صفحه‌ای که ده‌ها object دارد، روی چند اتصال فرستاده شود تا زودتر کامل شود؟

04-connection-tls.todo

- [>] اتصال در HTTP/1.x
- [ ] HTTP/2
- [ ] QUIC و HTTP/3
- [ ] TLS 1.3
- [ ] جمع‌بندی

اتصال را بعد از هر پاسخ ببندیم یا باز نگه داریم؟

- در non-persistent connection هر object اتصال تازه‌ای می‌گیرد و دو RTT هزینه دارد (Kurose و Ross، بخش ۲.۲.۲)
- در persistent connection، که پیش‌فرض HTTP/1.1 است، اتصال پس از پاسخ باز می‌ماند
- در pipelining درخواست‌ها بی‌انتظار فرستاده می‌شوند، اما پاسخ‌ها باید **به همان ترتیب** برگردند



سه مدل اتصال در HTTP/1.x، از چپ به راست

منبع: MDN, Mozilla Contributors · CC BY-SA 2.5

برقراری اتصال گران است، پس آن را باز نگه می‌داریم.

یک HTML و ده تصویر روی persistent connection چند RTT طول می‌کشد؟

زمان انتقال را ناچیز بگیرید. TLS و pipelining در کار نیست و مرورگر فقط یک اتصال باز می‌کند.

اول هر کس جداگانه رأی دهد؛ سپس دو دقیقه با بغل‌دستی خود بحث کنید و دوباره رأی دهید.

- الف) ۲
- ب) ۱۲
- ج) ۱۱
- د) ۲۲

یک HTML و ده تصویر روی persistent connection چند RTT طول می‌کشد؟

زمان انتقال را ناچیز بگیرید. TLS و pipelining در کار نیست و مرورگر فقط یک اتصال باز می‌کند.

اول هر کس جداگانه رأی دهد؛ سپس دو دقیقه با بغل‌دستی خود بحث کنید و دوباره رأی دهید.

الف) ۲ | ب) ۱۲ | ج) ۱۱ | د) ۲۲

پاسخ: ب) یک RTT برای برقراری اتصال، یکی برای HTML و یکی برای هر تصویر؛ روی هم ۱۲. با non-persistent connection این عدد ۲۲ می‌شود (گزینهٔ د) و با pipelining حدود ۳.

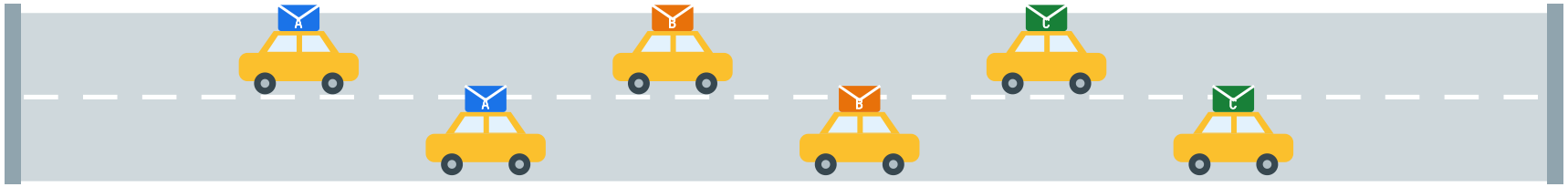
> **اتصال پایدار هزینهٔ handshake را حذف کرد، اما صف انتظار هنوز سر جایش است.**

در pipelining پاسخ‌ها به ترتیب برمی‌گردند؛ پس اگر پاسخ اول کند باشد، بقیه هم پشت سرش می‌مانند.

چرا مرورگر برای هر سرور تا شش اتصال هم‌زمان باز می‌کند؟

کلاینت

سرور



HTTP/1.1
شش اتصال؛ در هر اتصال، هر بار فقط یک پاسخ

> *Head-of-line blocking (HOL blocking) refers to a performance bottleneck that occurs when a queue of packets is held up by the first packet in the queue, even though other packets in the queue could be processed.*
>
> MDN, Glossary: Head-of-line blocking

- مرورگرها به همین دلیل pipelining را خاموش کردند و به‌جای آن برای هر host تا شش اتصال موازی باز می‌کنند؛ یعنی **شش برابر handshake**

مرورگر صف را با شش اتصال کوتاه‌تر کرد، اما مشکل را حل نکرد.

کدام headerها اتصال را کنترل می‌کنند؟

- سرور با `Connection: close` اعلام می‌کند که اتصال پس از این پاسخ بسته می‌شود، و با `Keep-Alive: timeout=5` پیشنهاد می‌دهد اتصال تا پنج ثانیه باز بماند (MDN)
- این دو فقط در HTTP/1.x معنا دارند؛ در HTTP/2 و HTTP/3 فرستادن headerهای مخصوص اتصال ممنوع است (RFC 9113 §8.2.2)
- در domain sharding منابع صفحه بین چند نام مثل `img1.example.com` پخش می‌شد تا مرورگر اتصال بیشتری باز کند؛ در HTTP/2 این ترفند فقط **handshake اضافه** می‌سازد

> **ترفندهای دوران HTTP/1.1 در HTTP/2 به ضرر تمام می‌شوند.**

بخش ۲ از ۵

HTTP/2: یک اتصال و چند stream

تا اینجا: در HTTP/1.1 مرورگر برای فرار از صف، شش اتصال هم‌زمان باز کرد.

سؤال این بخش

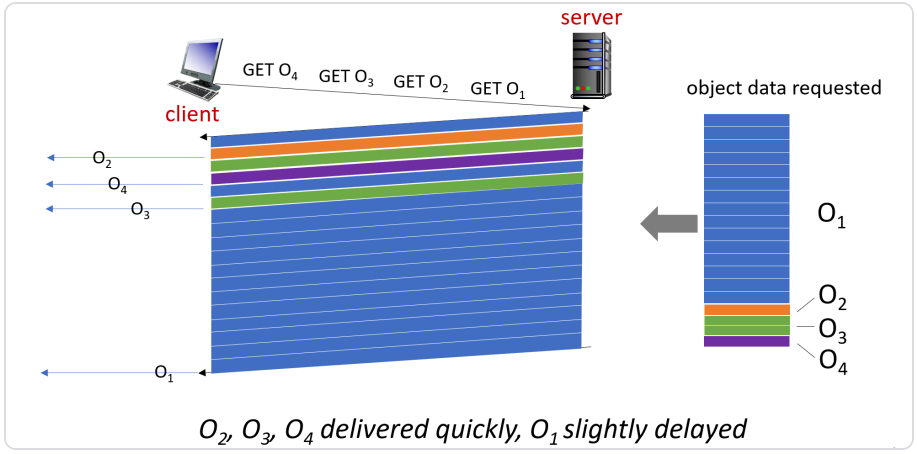
آیا می‌شود همهٔ پاسخ‌ها را روی یک اتصال فرستاد، بی‌آنکه منتظر یکدیگر بمانند؟

04-connection-tls.todo

- [x] اتصال در HTTP/1.x
- [>] **HTTP/2** ◀
- [ ] QUIC و HTTP/3
- [ ] TLS 1.3
- [ ] جمع‌بندی

چگونه چند پاسخ روی یک اتصال منتظر یکدیگر نمی‌مانند؟

- در HTTP/2 هر درخواست و پاسخِ آن یک stream با شناسهٔ خودش است و پیام به frameهای کوچک تقسیم می‌شود (قالب باینری را در جلسهٔ ۲ دیدیم)
- frameهای streamهای مختلف درهم فرستاده می‌شوند؛ به این کار **multiplexing** می‌گویند. حالا یک object بزرگ دیگر جلوی objectهای کوچک را نمی‌گیرد
- برای کل صفحه یک اتصال TCP و یک handshake کافی است



HTTP/2 تکه‌های object بزرگ را با objectهای کوچک درهم می‌فرستد

منبع: Kurose & Ross, Chapter 2 slides · author slides

HTTP/2 صف را در لایهٔ HTTP از میان برداشت.

یک بستهٔ TCP گم شده است؛ کدام streamها منتظر می‌مانند؟

سه تصویر A و B و C درهم روی یک اتصال HTTP/2 فرستاده می‌شوند و بستهٔ حامل B1 در راه گم می‌شود.



الف) فقط B

ب) هیچ‌کدام

ج) هر سه

د) فقط A و C

یک بستهٔ TCP گم شده است؛ کدام streamها منتظر می‌مانند؟

سه تصویر A و B و C درهم روی یک اتصال HTTP/2 فرستاده می‌شوند و بستهٔ حامل B1 در راه گم می‌شود.



الف) فقط B

ب) هیچ‌کدام

ج) هر سه

د) فقط A و C

پاسخ: ج) TCP از streamها خبر ندارد و همهٔ بایت‌ها را به ترتیب تحویل می‌دهد؛ پس تا B1 دوباره نرسد، A2 و C2 هم منتظر می‌مانند.

HTTP/2 مشکل HOL را در لایهٔ HTTP حل کرد، اما TCP آن را یک لایه پایین‌تر دوباره ساخت.

سناریوی واقعی: چرا Rapid Reset روی HTTP/1.1 کار نمی‌کند؟

- سرور HTTP/2 برای streamهای هم‌زمان هر اتصال سقفی می‌گذارد (مثلاً ۱۰۰)؛ اما streamی که کلاینت با `RST_STREAM` لغو کند **دیگر شمرده نمی‌شود**
- در تابستان ۱۴۰۲ مهاجمان با همین ترفند ۳۹۸ میلیون درخواست در ثانیه به Google فرستادند

الف) رمزنگاری ندارد	**ب)** لغو یعنی بستن اتصال	**ج)** سرورهایش قوی‌ترند	**د)** روی UDP است

سناریوی واقعی: چرا Rapid Reset روی HTTP/1.1 کار نمی‌کند؟

- سرور HTTP/2 برای streamهای هم‌زمان هر اتصال سقفی می‌گذارد (مثلاً ۱۰۰)؛ اما streamی که کلاینت با `RST_STREAM` لغو کند **دیگر شمرده نمی‌شود**
- در تابستان ۱۴۰۲ مهاجمان با همین ترفند ۳۹۸ میلیون درخواست در ثانیه به Google فرستادند

الف) رمزنگاری ندارد	**ب)** لغو یعنی بستن اتصال	**ج)** سرورهایش قوی‌ترند	**د)** روی UDP است

پاسخ: ب) در HTTP/2 اتصال باز می‌ماند و سرور کار هر درخواست را آغاز کرده بود، ولی سقف پر نمی‌شد. در HTTP/1.1 هر لغو یعنی بستن اتصال.

> **همان قابلیتی که صف را حذف کرد، سطح حمله را بزرگ‌تر کرد.**

چرا server push در HTTP/2 شکست خورد؟

- در server push سرور، پیش از آنکه مرورگر چیزی بخواهد، فایل‌هایی را می‌فرستاد که حدس می‌زد لازم می‌شوند. اما این فایل‌ها اغلب از قبل در cache مرورگر بودند و پهنای باند هدر می‌رفت
- مرورگر Chrome از نسخهٔ ۱۰۶ (مهر ۱۴۰۱) این قابلیت را حذف کرد (Chrome blog)
- جانشین آن پاسخ موقت `103 Early Hints` است: سرور فقط پیشنهاد می‌دهد و **تصمیم با مرورگر است**

> **سرور نمی‌داند در cache مرورگر چه هست؛ پس تصمیم را باید کسی بگیرد که می‌داند.**

بخش ۳ از ۵

QUIC و HTTP/3

تا اینجا: HTTP/2 صف را در لایهٔ HTTP برداشت، اما TCP هنوز همهٔ بایت‌ها را به ترتیب تحویل می‌دهد.

سؤال این بخش

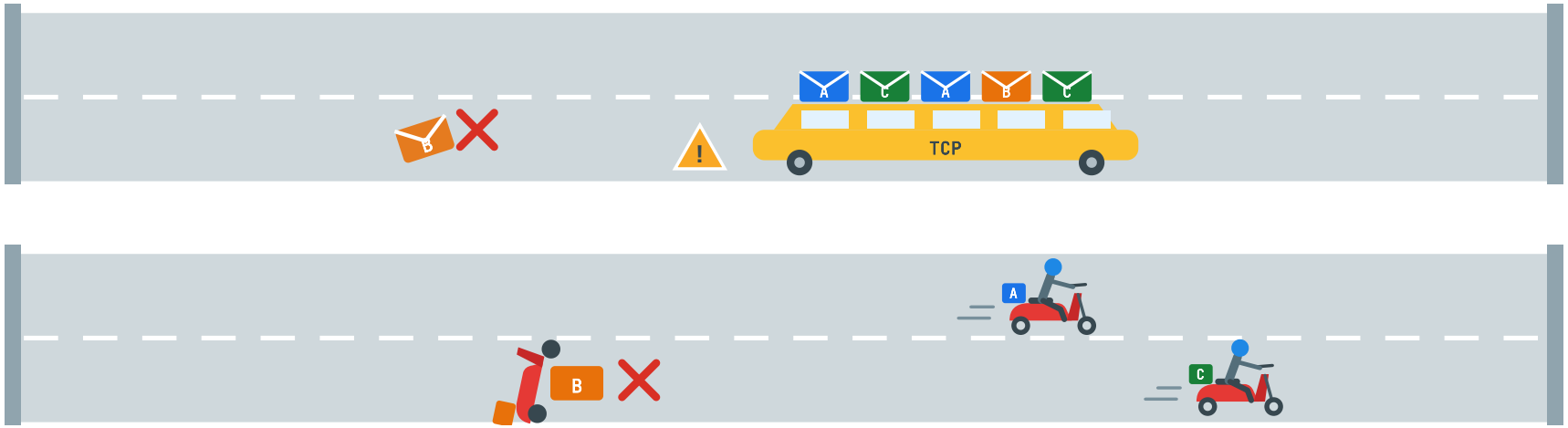
اگر نمی‌توانیم TCP را عوض کنیم، مسئله را در کجا حل کنیم؟

04-connection-tls.todo

- [x] اتصال در HTTP/1.x
- [x] HTTP/2
- [>] QUIC و HTTP/3 ◀
- [ ] TLS 1.3
- [ ] جمع‌بندی

QUIC چگونه صف را از لایهٔ انتقال برداشت؟

کلاینت سرور



> *QUIC avoids this by running over UDP and implementing packet loss detection and retransmission separately for each stream, meaning that packet loss only blocks the particular stream whose packets were lost.*
>
> MDN, Glossary: QUIC

QUIC صف را به لایهٔ انتقال برد؛ همان جایی که مشکل بود.

درخواست‌های این صفحه با کدام نسخهٔ HTTP فرستاده شده‌اند؟

۱ در DevTools به زبانهٔ Network بروید، روی سرستون‌ها راست‌کلیک کنید و ستون Protocol را فعال کنید

۲ یک سایت بزرگ را دو بار بارگذاری کنید و بشمارید چند درخواست `http/1.1` و `h2` و `h3` هستند

۳ چرا یک درخواست در بار اول `h2` است و در بار دوم `h3` ؟

درخواست‌های این صفحه با کدام نسخهٔ HTTP فرستاده شده‌اند؟

۱ در DevTools به زبانهٔ Network بروید، روی سرستون‌ها راست‌کلیک کنید و ستون Protocol را فعال کنید

۲ یک سایت بزرگ را دو بار بارگذاری کنید و بشمارید چند درخواست `http/1.1` و `h2` و `h3` هستند

۳ چرا یک درخواست در بار اول `h2` است و در بار دوم `h3` ؟

پاسخ: سرور در پاسخ HTTP/2 با header `Alt-Svc` اعلام می‌کند که HTTP/3 را هم پشتیبانی می‌کند، و مرورگر از بار بعد آن را امتحان می‌کند. رکورد HTTPS در DNS (جلسهٔ ۱) همین را از بار اول می‌گوید. اگر UDP در شبکه بسته باشد، مرورگر بی‌صدا به HTTP/2 برمی‌گردد.

> **HTTP/3 همیشه یک راه بازگشت به HTTP/2 دارد.**

فعال کردن HTTP/3 برای سایت دانشکده می‌ارزد؟

حدود ۳۹٪ سایت‌ها HTTP/3 را اعلام کرده‌اند (W3Techs)، اما فقط حدود ۲۰٪ درخواست‌های Cloudflare با آن فرستاده می‌شود (تابستان ۱۴۰۵).

الف) بله، همیشه

ب) بله، برای کاربران اینترنت همراه

ج) نه، UDP همه‌جا بسته است

د) نه، HTTP/2 کافی است

فعال کردن HTTP/3 برای سایت دانشکده می‌ارزد؟

حدود ۳۹٪ سایت‌ها HTTP/3 را اعلام کرده‌اند (W3Techs)، اما فقط حدود ۲۰٪ درخواست‌های Cloudflare با آن فرستاده می‌شود (تابستان ۱۴۰۵).

الف) بله، همیشه	**ب)** بله، برای کاربران اینترنت همراه	**ج)** نه، UDP همه‌جا بسته است	**د)** نه، HTTP/2 کافی است

پاسخ: ب) سود HTTP/3 بیشتر در شبکه‌های پرتلفات و هنگام جابه‌جایی بین شبکه‌هاست؛ هزینه‌اش پردازش بیشتر و باز کردن UDP روی درگاه ۴۴۳ است.

> **HTTP/3 جایگزین HTTP/2 نیست؛ مسیر سریع‌تری در کنار آن است.**

چرا QUIC روی UDP ساخته شد، و چرا با عوض شدن شبکه قطع نمی‌شود؟

- TCP در هستهٔ سیستم‌عامل‌ها و در تجهیزات میانی شبکه پیاده شده و تغییر دادنش سال‌ها طول می‌کشد. اما QUIC را خود مرورگر و سرور، روی UDP، پیاده می‌کنند
- هر اتصال TCP با چهار مقدار شناخته می‌شود: IP و port دو طرف. اگر IP عوض شود، اتصال از بین می‌رود و همهٔ handshakeها باید از نو انجام شوند
- در QUIC هر اتصال یک connection ID دارد. کلاینت از نشانی تازه ادامه می‌دهد و سرور پیش از فرستادن دادهٔ زیاد، مسیر تازه را می‌آزماید؛ به این کار connection migration می‌گویند

> **اتصال TCP به نشانی گره خورده است و اتصال QUIC به شناسه.**

تا اینجا فقط یک محور را دنبال کردیم: فرستادن چند پیام هم‌زمان. هر نسخه چیزهای دیگری هم آورد.

از HTTP/0.9 تا HTTP/3: هر نسخه چه چیزی اضافه کرد؟

نسخه	سال	سند امروز	انتقال	آنچه اضافه شد
HTTP/0.9	۱۳۷۰	بدون RFC	اتصال TCP برای هر درخواست	`GET` only, HTML only; no headers, no status codes
HTTP/1.0	۱۳۷۵	RFC 1945	اتصال TCP برای هر درخواست	headers, status codes, `Content-Type`, `POST`, `HEAD`
HTTP/1.1	۱۳۷۵	RFC 9112	persistent connection و pipelining	`Host`, chunked encoding, `Cache-Control`, `ETag`, ranges
HTTP/2	۱۳۹۴	RFC 9113	یک اتصال TCP با frame باینری	multiplexing, HPACK, server push (later dropped)
HTTP/3	۱۴۰۱	RFC 9114	QUIC روی UDP با TLS 1.3 درونی	independent streams, QPACK, 0-RTT, migration

هر نسخه محدودیتی از نسخهٔ قبل را برداشت؛ ارسال هم‌زمان فقط یکی از آن‌ها بود.

هر نسخه محدودیتی را برداشت و هزینه‌ای تازه آورد؛ پس انتخاب نسخه یک تصمیم مهندسی است، نه یک ارتقای خودکار.

در چه شرایطی کدام نسخهٔ HTTP را انتخاب کنیم؟

شرایط	انتخاب	دلیل
سرویس داخلی، ابزار خط فرمان یا تجهیز قدیمی در مسیر	`HTTP/1.1`	متنی و ساده است، همه‌جا پشتیبانی می‌شود و با `curl` به‌راحتی اشکال‌یابی می‌شود
سایت مرورگرمحور با ده‌ها object روی HTTPS	`HTTP/2`	یک اتصال و multiplexing؛ مرورگرها آن را عملاً فقط روی TLS به کار می‌برند
کاربران موبایل، شبکهٔ پرتلفات یا جابه‌جایی بین Wi-Fi و شبکهٔ همراه	`HTTP/3` در کنار `HTTP/2`	هر stream مستقل است، connection ID اتصال را نگه می‌دارد و بازگشت به HTTP/2 همیشه ممکن است
UDP روی درگاه ۴۴۳ بسته است یا پردازش سرور محدود است	`HTTP/2`	QUIC پردازش بیشتری می‌خواهد و مرورگر خودش به HTTP/2 برمی‌گردد

سه پرسش پشت هر انتخاب: کلاینت و مسیر چه نسخه‌ای را می‌فهمند؟ شبکهٔ کاربر چقدر RTT و تلفات دارد؟ سرور چه هزینه‌ای می‌پردازد؟

> **نسخه را شبکهٔ کاربر و هزینهٔ سرور تعیین می‌کنند، نه تازه بودن پروتکل.**

بخش ۴ از ۵

TLS 1.3: کلید مشترک بدون دیدار قبلی

تا اینجا: در جلسهٔ ۱ دیدیم که TLS سه چیز را تضمین می‌کند: محرمانگی، یکپارچگی و اصالت سرور.

سؤال این بخش

دو طرفی که هرگز همدیگر را ندیده‌اند، روی شبکه‌ای که همه آن را می‌شنوند، چگونه بر یک کلید رمز توافق می‌کنند؟

04-connection-tls.todo

- [x] اتصال در HTTP/1.x
- [x] HTTP/2
- [x] QUIC و HTTP/3
- [>] **TLS 1.3** ▶
- [ ] جمع‌بندی

در TLS 1.3 کلید رمز چگونه بین مرورگر و سرور مشترک می‌شود؟

مرورگر و سرور هیچ راز مشترکی ندارند و ناظر شبکه همهٔ پیام‌هایشان را می‌بیند.

الف) مرورگر آن را بی‌رمز می‌فرستد

ب) همراه گواهی سرور می‌آید

ج) هر طرف آن را خودش می‌سازد

د) از پیش در مرورگر هست

در TLS 1.3 کلید رمز چگونه بین مرورگر و سرور مشترک می‌شود؟

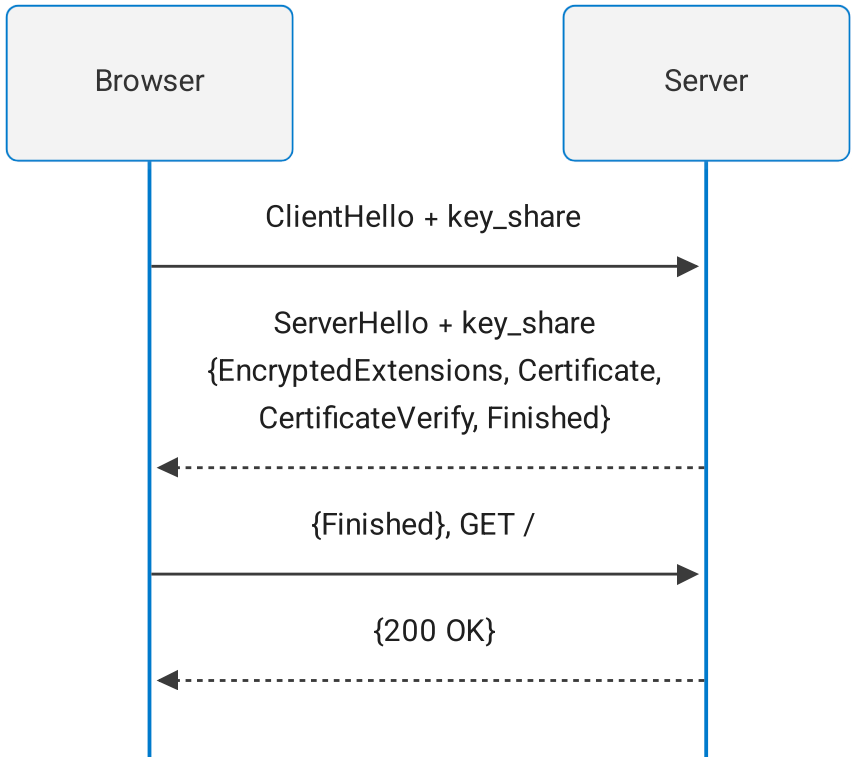
مرورگر و سرور هیچ راز مشترکی ندارند و ناظر شبکه همهٔ پیام‌هایشان را می‌بیند.

الف) مرورگر آن را بی‌رمز می‌فرستد	**ب)** همراه گواهی سرور می‌آید	**ج)** هر طرف آن را خودش می‌سازد	**د)** از پیش در مرورگر هست

پاسخ: ج) در Diffie–Hellman هر طرف عددی مخفی نگه می‌دارد و فقط حاصل یک محاسبهٔ یک‌طرفه را می‌فرستد؛ ناظر از این دو پیام نمی‌تواند کلید را حساب کند. در TLS 1.2 کلید را می‌شد با کلید عمومی سرور رمز کرد و فرستاد، که TLS 1.3 آن را حذف کرد.

> **در handshake کلید ساخته می‌شود، نه فرستاده.**

فرایند handshake در TLS 1.3 در یک RTT چه می‌کند؟



- هر طرف یک `key_share` می‌فرستد و از این دو، کلید مشترک را می‌سازد. **پیام‌های داخل آکولاد** با همین کلید رمز شده‌اند
- گواهی و امضای `CertificateVerify` ثابت می‌کنند که سرور صاحب همان نامی است که در URL آمده (RFC 8446)

گواهی کلید نمی‌سازد؛ فقط نشان می‌دهد کلید را با چه کسی ساخته‌اید.

هر نسل پروتکل تا اولین بایت پاسخ چند RTT لازم دارد؟

پشتهٔ پروتکل	handshake اتصال	handshake TLS	Request / Response	جمع RTT
TCP + TLS 1.2	۱	۲	۱	**۴**
TCP + TLS 1.3	۱	۱	۱	**۳**
QUIC (HTTP/3)	۱: اتصال و TLS با هم		۱	**۲**

- handshake کامل در TLS 1.2 دو RTT می‌گرفت. TLS 1.3 با فرستادن `key_share` در پیام اول یک RTT را حذف کرد، مگر سرور گروه کلید دیگری بخواهد (HelloRetryRequest)
- در QUIC هر دو handshake، یعنی اتصال و TLS، در یک رفت‌وبرگشت انجام می‌شوند

> **هر نسل دست‌کم یک رفت‌وبرگشت را حذف کرده است.**

در TLS 1.3، کدام درخواست‌ها را در `0-RTT` بپذیریم؟

> *Early data (also known as zero round-trip time (0-RTT) data) allows the client to send data to a server in the first round trip of a connection, without waiting for the TLS handshake to complete.*
>
> MDN, 425 Too Early

این امکان فقط در بازگشت به سرور آشناست؛ اما مهاجم می‌تواند همان بسته را دوباره بفرستد.

- **الف)** همه را
- **ب)** فقط safeها را
- **ج)** فقط POSTها را
- **د)** هیچ‌کدام را

در TLS 1.3، کدام درخواست‌ها را در `0-RTT` بپذیریم؟

> *Early data (also known as zero round-trip time (0-RTT) data) allows the client to send data to a server in the first round trip of a connection, without waiting for the TLS handshake to complete.*
>
> MDN, 425 Too Early

این امکان فقط در بازگشت به سرور آشناست؛ اما مهاجم می‌تواند همان بسته را دوباره بفرستد.

الف) همه را	ب) فقط safeها را	ج) فقط POSTها را	د) هیچ‌کدام را

پاسخ: ب) رمزنگاری جلوی تکرار را نمی‌گیرد؛ سرور فقط درخواست safe مثل `GET /news` را می‌پذیرد و به بقیه `425` می‌دهد (RFC 8470).

رمزنگاری جلوی خواندن را می‌گیرد، نه جلوی تکرار را.

> Q-Day روزی است که الگوریتم Shor روی کلیدهای واقعی اجرا شود؛ زمان آن را هنوز کسی نمی‌داند.

ترافیک TLS ضبط‌شدهٔ امروز را رایانهٔ کوانتومی فردا باز می‌کند؟

الف) نه؛ AES شکسته نمی‌شود

ب) بله؛ کلید بازسازی می‌شود

ج) نه؛ forward secrecy محافظ است

د) بله؛ AES شکسته می‌شود

Q-Day روزی است که الگوریتم Shor روی کلیدهای واقعی اجرا شود؛ زمان آن را هنوز کسی نمی‌داند.

ترافیک TLS ضبط‌شدهٔ امروز را رایانهٔ کوانتومی فردا باز می‌کند؟

الف) نه؛ AES شکسته نمی‌شود

ب) بله؛ کلید بازسازی می‌شود

ج) نه؛ forward secrecy محافظ است

د) بله؛ AES شکسته می‌شود

پاسخ: ب) Shor امنیت X25519 را می‌شکند؛ پس هر `key_share` ضبط‌شده بعدها کلید AES را لو می‌دهد (harvest now, decrypt later). forward secrecy جلوی لو رفتن کلید سرور را می‌گیرد، نه حل شدن Diffie–Hellman.

رمزنگاری امروز را می‌شود برای رمزگشایی فردا ضبط کرد.

چگونه تبادل کلید TLS را در برابر رایانهٔ کوانتومی ایمن کنیم؟

- در تبادل کلید ترکیبی (hybrid)، هر دو الگوریتم X25519 و ML-KEM-768 اجرا می‌شوند و کلید از هر دو ساخته می‌شود؛ مهاجم باید **هر دو را بشکند**
- ML-KEM استاندارد NIST است (FIPS 203، مرداد ۱۴۰۳) و RFC 10024 گروه `X25519MLKEM768` را برای TLS 1.3 ثبت کرده است
- مرورگرها از پاییز ۱۴۰۳ آن را پیش‌فرض کرده‌اند و حدود ۵۲٪ ترافیک انسانی Cloudflare در ۱۴۰۴ با آن رمز شد (Cloudflare Radar)؛ طرف قدیمی‌تر به `X25519` برمی‌گردد

گروه	key_share مرورگر	key_share سرور
`X25519`	۳۲ بایت	۳۲ بایت
`X25519MLKEM768`	۱۲۱۶ بایت	۱۱۲۰ بایت

هزینهٔ امنیت پساکوانتومی چند صد بایت بیشتر در handshake است، نه رفت‌وبرگشت بیشتر.

> **اگر داده باید سال‌ها محرمانه بماند، تبادل کلید را به Q-Day نسپارید.**

پیوند با هوش مصنوعی: سایت از کجا می‌فهمد که کلاینتش یک agent است؟

- پیام ClientHello رمز نشده است. فهرست نسخه‌ها، الگوریتم‌ها و extensionهای آن در هر کتابخانهٔ TLS شکل خاص خودش را دارد، و به خلاصهٔ آن JA4 fingerprint می‌گویند
- پژوهشی در سال ۱۴۰۵ شش agent مبتنی بر LLM را بررسی کرد. با ترکیب IP و TLS fingerprint و ویژگی‌های مرورگر، **همهٔ آن‌ها از کاربر انسانی تشخیص داده شدند**؛ agentهایی که در cloud اجرا می‌شدند بیشتر از روی IP و JA4 شناسایی شدند

> **handshake پیش از آنکه رمز شود، امضای نرم‌افزار کلاینت را آشکار می‌کند.**

TLS 1.3 چه چیزهایی را از TLS 1.2 حذف کرد؟

- در TLS 1.2 مرورگر می‌توانست کلید را با کلید عمومی RSA سرور رمز کند و بفرستد. در این حالت هر کس بعدها کلید خصوصی سرور را به دست آورد، همهٔ ترافیک ضبط‌شدهٔ گذشته را می‌تواند باز کند
- در TLS 1.3 تبادل کلید با RSA حذف شد و handshake کامل از Diffie–Hellman گذرا (ephemeral) استفاده می‌کند: کلید هر اتصال پس از پایان آن دور ریخته می‌شود. به این ویژگی forward secrecy می‌گویند؛ دادهٔ `0-RTT` و حالت PSK بدون Diffie–Hellman آن را ندارند
- الگوریتم‌های قدیمی، فشرده‌سازی و renegotiation هم حذف شدند و handshake از ServerHello به بعد رمز می‌شود (RFC 8446 §1.2)

> **در TLS 1.3 دزدیدن کلید سرور، ترافیک گذشته را آشکار نمی‌کند.**

امضای گواهی در برابر رایانهٔ کوانتومی چه می‌شود؟

- تبادل کلید فوری است، چون ضبط امروز را تهدید می‌کند. امضای گواهی را مهاجم باید **هنگام اتصال** جعل کند، پس مهلت مهاجرتش بیشتر است
- امضاهای ML-DSA (FIPS 204) و SLH-DSA (FIPS 205) بزرگ‌اند: بار احراز اصالت یک handshake با ML-DSA-44 از حدود ۱٫۲ به حدود ۱۵ کیلوبایت می‌رسد (گزارش)
- پاسخ در دست آزمایش، Merkle Tree Certificates است: CA فقط ریشهٔ یک درخت را امضا می‌کند و گواهی، اثبات عضویت با چند hash است

> **کلید را همین امروز باید عوض کرد؛ امضا را می‌شود با مهلت بیشتری.**

یادآوری جلسهٔ ۳: کوکی نشست همان قدرتی را دارد که رمز عبور.

ویژگی Secure کوکی چه ارتباطی با TLS دارد؟

- کوکی‌ای که `Secure` دارد فقط روی https فرستاده می‌شود. بدون آن، **یک درخواست http کافی است** تا کوکی نشست بدون رمز در شبکه منتقل شود
- TLS کوکی را فقط در مسیر انتقال پنهان می‌کند؛ نه در مرورگر و نه در فایل HAR (نفوذ Okta در جلسهٔ ۳)
- HSTS و فهرست preload آن (جلسهٔ ۱) جلوی همان اولین درخواست http را هم می‌گیرند

ویژگی `Secure` تضمین می‌کند که کوکی فقط از مسیر رمزشده عبور کند.

اگر یک CA به اشتباه گواهی صادر کند، چه کسی متوجه می‌شود؟

- مرورگرهای Chrome و Safari و Firefox (نسخهٔ دسکتاپ از ۱۳۵) گواهی CAهای عمومی را فقط وقتی می‌پذیرند که در لاگ‌های Certificate Transparency (CT) ثبت شده باشد
- صاحب دامنه می‌تواند این لاگ‌ها را جستجو کند، مثلاً در crt.sh، و **گواهی‌های ناخواسته** برای دامنهٔ خود را پیدا کند
- رکورد CAA در DNS مشخص می‌کند کدام CAها اجازهٔ صدور گواهی برای این دامنه را دارند

> **لاگ‌های CT جلوی صدور اشتباه را نمی‌گیرند، اما آن را آشکار می‌کنند.**

بخش ۵ از ۵

جمع‌بندی

تا اینجا: در این جلسه دیدیم هر نسل HTTP و TLS کدام زمان انتظار را حذف کرد.

سؤال این بخش

یک درخواست تازه، در هر نسل، چند RTT طول می‌کشد تا اولین بایت پاسخ برسد؟

04-connection-tls.todo

- [x] اتصال در HTTP/1.x
- [x] HTTP/2
- [x] QUIC و HTTP/3
- [x] TLS 1.3
- [>] جمع‌بندی ▶

جدول RTT را از حافظه کامل کنید

پشتهٔ پروتکل	handshake اتصال	handshake TLS	Request / Response	جمع RTT
TCP + TLS 1.2	؟	؟	؟	؟
TCP + TLS 1.3	۱	۱	۱	**۳**
TCP + TLS 1.3 + 0-RTT	؟	؟	؟	؟
QUIC (HTTP/3)	؟		؟	؟
QUIC + 0-RTT	؟		؟	؟

اول ردیف‌ها را از حافظه بگویید، سپس با پاسخ مقایسه کنید.

جدول RTT را از حافظه کامل کنید

پشتهٔ پروتکل	handshake اتصال	handshake TLS	Request / Response	جمع RTT
TCP + TLS 1.2	۱	۲	۱	**۴**
TCP + TLS 1.3	۱	۱	۱	**۳**
TCP + TLS 1.3 + 0-RTT	۱	۰: درخواست همراه ClientHello	۱	**۲**
QUIC (HTTP/3)	۱: اتصال و TLS با هم		۱	**۲**
QUIC + 0-RTT	۰: درخواست همراه اولین پیام		۱	**۱**

اول ردیف‌ها را از حافظه بگویید، سپس با پاسخ مقایسه کنید.

تاریخچهٔ این جلسه، تاریخچهٔ حذف رفت‌وبرگشت‌هاست.

این مفاهیم کدام تصمیم‌های مهندسی را بهتر می‌کنند؟

مدل ذهنی	تصمیم مهندسی
زمان را تعداد RTTها تعیین می‌کند	سرور را به کاربر نزدیک کنید و اتصال را دوباره به کار ببرید
پاسخ‌های HTTP/1.1 به ترتیب برمی‌گردند	HTTP/2 را فعال کنید و domain sharding را کنار بگذارید
TCP همهٔ بایت‌ها را به ترتیب تحویل می‌دهد	برای شبکه‌های پرتلفات، HTTP/3 را هم فعال کنید
دادهٔ `0-RTT` ممکن است تکرار شود	در `0-RTT` فقط درخواست safe را بپذیرید
ترافیک ضبط‌شده بعدها باز می‌شود	گروه کلید ترکیبی `X25519MLKEM768` را فعال کنید

جلسهٔ بعد: سریع‌ترین درخواست آن است که اصلاً به سرور نرسد؛ cache و CDN.

تمرین ۲، گام ۳: همان سرور، روی TLS

چالش سرور کیف پول بوفه را، که در گام‌های ۱ و ۲ روی سوکت خام ساختید، با گزینهٔ `--tls` روی TLS اجرا کنید. بعد ترافیک ورود را یک بار بدون TLS و یک بار با TLS ضبط کنید، و زمان هر مرحلهٔ یک درخواست به یک سایت واقعی را با `curl` بسنجید.

سؤال درخواست دوم به همان سرور، روی همان اتصال، چند RTT کمتر از درخواست اول لازم دارد؟

پل درخواست دوم برای یک تصویر از همان سرور شاید اصلاً به شبکه نرود؛ جلسهٔ ۵ دربارهٔ cache است.

تا جمعه ۸ آبان، ۲۳:۵۹

ie1405-scu.github.io/exercises/02-socket-to-tls/

نقشهٔ فصل ۱: کجای راهیم؟

ch1/syllabus.todo

- [x] جلسهٔ ۱: از URL تا سرور — URL و Origin و DNS، و TLS از دور
- [x] جلسهٔ ۲: یک تراکنش HTTP — متد و کد وضعیت و header
- [x] جلسهٔ ۳: state و هویت — cookie و CORS و CSRF
- [>] **جلسهٔ ۴: زیربنای HTTP — TCP و QUIC و TLS** ▶
- [ ] جلسهٔ ۵: واسطه‌ها — cache و proxy و CDN
- [ ] جلسهٔ ۶: فراتر از request/response — SSE و WebSocket
- [ ] جلسهٔ ۷: کلاینت‌های خودکار — agent و MCP

هر جلسه یک حلقه از همان زنجیرهٔ ابتدای فصل را بررسی می‌کند.

برای کنجکاوی بیشتر (۱ از ۲)

۱ چرا QUIC بخشی از header بسته‌ها را هم رمز می‌کند، حتی شمارهٔ بسته را؟

سرنخ: دربارهٔ protocol ossification بخوانید، و بخش header protection در RFC 9001 §5.4 را ببینید.

۲ قرار بود TCP Fast Open داده را همراه بستهٔ SYN بفرستد؛ چرا هیچ‌وقت رایج نشد؟

سرنخ: RFC 7413 را ببینید، و رفتار middleboxهایی را که بستهٔ SYN حامل داده را دور می‌اندازند.

۳ چگونه می‌توان نام سایت را هم از ClientHello پنهان کرد، و چرا این کار به DNS وابسته است؟

سرنخ: دربارهٔ Encrypted Client Hello (ECH) در IETF، و رکورد HTTPS در RFC 9460 بخوانید.

برای کنجکاوی بیشتر (۲ از ۲)

۴ چرا ClientHello با `key_share` ترکیبی پساکوانتومی دیگر در یک بستهٔ QUIC Initial جا نمی‌شود، و این برای تجهیزاتی که SNI را از اولین بسته می‌خوانند چه مشکلی می‌سازد؟

سرنخ: دربارهٔ اندازهٔ دست‌کم ۱۲۰۰ بایتی دیتاگرام Initial در RFC 9000 §14.1 بخوانید، و ClientHello را در Wireshark روی یک اتصال HTTP/3 ببینید.

۵ چرا `curl --http3` در بسیاری از توزیع‌های Linux خطا می‌دهد؟

سرنخ: خروجی `curl -V` و فهرست Features را ببینید؛ curl با کدام کتابخانهٔ QUIC ساخته شده است؟

۶ وقتی همهٔ objectها روی یک اتصال‌اند، مرورگر چگونه تعیین می‌کند کدام زودتر برسد؟

سرنخ: header `Priority` را در RFC 9218 ببینید.

منابع

- منبع درسی: کتاب *Computer Networking* نوشتهٔ Kurose و Ross، بخش‌های ۲.۲.۲ و ۲.۲.۶ و ۳.۸ و ۸.۶
- RFC 9112 §9: Connection Management · RFC 9113: HTTP/2
- RFC 9000: QUIC · RFC 9001 · RFC 9114: HTTP/3
- RFC 8446: TLS 1.3 · RFC 8470: Early Data
- MDN: Connection management in HTTP/1.x
- MDN: Head-of-line blocking · QUIC · Alt-Svc
- Belshe: More Bandwidth Doesn't Matter (much)
- Google Cloud: HTTP/2 Rapid Reset
- Chrome: Removing HTTP/2 Server Push
- Cloudflare Radar · W3Techs: HTTP/3
- arXiv 2606.30119: Unmasking Web Agents · JA4
- MDN: Certificate Transparency
- RFC 10024: Hybrid ML-KEM for TLS 1.3 · NIST: FIPS 203 و 204 و 205
- Gidney: RSA-2048 با کمتر از یک میلیون کیوبیت · Merkle Tree Certificates